

1. Código completo

js/app.js

Reemplazá todo el contenido de app.js por esto:

```
import { descargarProductosTech } from "../api.js";
import {
  obtenerFavoritos,
  alternarFavorito
} from "../storage.js";
import {
  crearTarjetaProductoHTML,
  filtrarProductos,
  calcularTotalCatalogo
} from "../ui.js";

// =====
// 1. SELECCIÓN DE ELEMENTOS DEL DOM
// =====

const btnTema = document.querySelector("#btn-tema");
const iconoTema = document.querySelector("#icono-tema");

const btnVerFavoritos = document.querySelector("#btn-ver-favoritos");
const badgeFavoritos = document.querySelector("#badge-favoritos-contador");

const inputBuscador = document.querySelector("#input-buscador");
const botonesFiltro = document.querySelectorAll(".btn-filtro");

const contenedorCatalogo = document.querySelector("#contenedor-catalogo");

const totalProductosSpan = document.querySelector("#total-productos-visibles");
const totalPrecioSpan = document.querySelector("#total-precio-acumulado");
```

```
const estadoLoading = document.querySelector("#estado-loading");
const estadoError = document.querySelector("#estado-error");
const btnReintentar = document.querySelector("#btn-reintentar");
const sinResultadosBox = document.querySelector("#sin-
resultados");
```

```
// =====
// 2. ESTADO GLOBAL
// =====
```

```
let productosEnMemoria = [];
let categoriaActual = "todas";
```

```
// =====
// 3. ACTUALIZAR CONTADOR DE FAVORITOS
// =====
```

```
function actualizarBadgeFavoritos() {

    const favoritos = obtenerFavoritos();

    badgeFavoritos.textContent = favoritos.length;
}
```

```
// =====
// 4. APLICAR FILTROS
// =====
```

```
function aplicarFiltros() {

    const texto = inputBuscador.value.trim();

    const favoritosIds = obtenerFavoritos();

    const productosFiltrados = filtrarProductos(
        productosEnMemoria,
        texto,
        categoriaActual,
```

```

        favoritosIds
    );

    totalProductosSpan.textContent = productosFiltrados.length;

    const totalPrecio =
    calcularTotalCatalogo(productosFiltrados);

    totalPrecioSpan.textContent = `${totalPrecio.toFixed(2)}`;

    if (productosFiltrados.length === 0) {

        contenedorCatalogo.innerHTML = "";

        sinResultadosBox.classList.remove("oculto");

    } else {

        sinResultadosBox.classList.add("oculto");

        contenedorCatalogo.innerHTML = productosFiltrados
            .map(producto => {

                const esFavorito =
                favoritosIds.includes(producto.id);

                return crearTarjetaProductoHTML(
                    producto,
                    esFavorito
                );

            })
            .join("");
    }
}

```

```

// =====
// 5. ACTIVAR FILTRO DE CATEGORIA
// =====

```

```

function activarFiltroCategoria(categoria) {

    categoriaActual = categoria;

    botonesFiltro.forEach(boton => {

        const categoriaBoton = boton.dataset.categoria;

        if (categoriaBoton === categoria) {

            boton.classList.add("activo");

        } else {

            boton.classList.remove("activo");
        }
    });

    aplicarFiltros();
}

// =====
// 6. CARGAR CATALOGO DESDE LA API
// =====

async function cargarCatalogo() {

    estadoLoading.classList.remove("oculto");

    estadoError.classList.add("oculto");

    contenedorCatalogo.innerHTML = "";

    try {

        productosEnMemoria = await descargarProductosTech();

        estadoLoading.classList.add("oculto");

        aplicarFiltros();
    }
}

```

```

    } catch (error) {

        console.error("Error al cargar el catálogo:", error);

        estadoLoading.classList.add("oculto");

        estadoError.classList.remove("oculto");
    }
}

// =====
// 7. MODO OSCURO
// =====

btnTema.addEventListener("click", () => {

    document.body.classList.toggle("dark-mode");

    if (document.body.classList.contains("dark-mode")) {

        iconoTema.textContent = "☀️";

    } else {

        iconoTema.textContent = "🌙";
    }
});

// =====
// 8. BOTON DE FAVORITOS
// =====

btnVerFavoritos.addEventListener("click", () => {

    activarFiltroCategoria("favoritos");
});

// =====
// 9. BOTONES DE CATEGORIA

```

```

// =====

botonesFiltro.forEach(boton => {

    boton.addEventListener("click", () => {

        const categoria = boton.dataset.categoria;

        activarFiltroCategoria(categoria);
    });
});

// =====
// 10. BUSQUEDA EN TIEMPO REAL
// =====

inputBuscador.addEventListener("input", () => {

    aplicarFiltros();
});

// =====
// 11. BOTON DE REINTENTO
// =====

btnReintentar.addEventListener("click", () => {

    cargarCatalogo();
});

// =====
// 12. DELEGACION DE EVENTOS PARA FAVORITOS
// =====

contenedorCatalogo.addEventListener("click", evento => {

    const botonFavorito = evento.target.closest(".btn-fav-
card");

```

```

    if (!botonFavorito) {

        return;
    }

    const id = Number(botonFavorito.dataset.id);

    alternarFavorito(id);

    actualizarBadgeFavoritos();

    aplicarFiltros();
});

// =====
// 13. INICIALIZACION
// =====

actualizarBadgeFavoritos();

cargarCatalogo();

```

2. js/api.js

```

async function descargarProductosTech() {

    const urls = [
        "https://dummyjson.com/products/category/smartphones",
        "https://dummyjson.com/products/category/laptops",
        "https://dummyjson.com/products/category/mobile-accessories"
    ];

    const respuestas = await Promise.all(
        urls.map(url => fetch(url))
    );

    respuestas.forEach(respuesta => {

        if (!respuesta.ok) {

```

```

        throw new Error(
            `Error HTTP: ${respuesta.status}`
        );
    }
});

const datos = await Promise.all(
    respuestas.map(respuesta => respuesta.json())
);

const productos = datos.flatMap(
    datosCategoria => datosCategoria.products
);

return productos;
}

export {
    descargarProductosTech
};

```

3. js/storage.js

```

const FAVORITOS_KEY = "techstore_favoritos_v1";

function obtenerFavoritos() {

    const favoritosGuardados =
        localStorage.getItem(FAVORITOS_KEY);

    if (!favoritosGuardados) {

        return [];
    }

    try {

        return JSON.parse(favoritosGuardados);
    }
}

```

```
    } catch (error) {  
  
        console.error(  
            "Error al leer los favoritos:",  
            error  
        );  
  
        return [];  
    }  
}
```

```
function esProductoFavorito(id) {  
  
    const favoritos = obtenerFavoritos();  
  
    return favoritos.includes(id);  
}
```

```
function alternarFavorito(id) {  
  
    const favoritos = obtenerFavoritos();  
  
    const posicion = favoritos.indexOf(id);  
  
    if (posicion !== -1) {  
  
        favoritos.splice(posicion, 1);  
  
    } else {  
  
        favoritos.push(id);  
    }  
  
    localStorage.setItem(  
        FAVORITOS_KEY,  
        JSON.stringify(favoritos)  
    );  
}
```

```
export {
  obtenerFavoritos,
  esProductoFavorito,
  alternarFavorito
};
```

4. js/ui.js

```
import { esProductoFavorito } from "../storage.js";

function crearTarjetaProductoHTML(producto, esFavorito) {

  const {
    id,
    title,
    price,
    category,
    thumbnail,
    stock
  } = producto;

  return `
    <article class="tarjeta-producto" data-id="${id}">

      <div class="tarjeta-img-wrap">

        <span class="badge-categoria">
          ${category}
        </span>

        <button
```

```

        class="btn-fav-card ${esFavorito ? "en-
favoritos" : ""}"
        data-id="${id}"
        aria-label="Guardar favorito"
    >
        ${esFavorito ? "★" : "☆"}
    </button>

</div>

<div class="tarjeta-cuerpo">

    <h3 class="tarjeta-titulo">
        ${title}
    </h3>

    <div class="tarjeta-precio-wrap">

        <span class="tarjeta-precio">
            ${price.toFixed(2)}
        </span>

        <span class="tarjeta-stock">
            Stock: ${stock}
        </span>

    </div>

</div>

</article>
`;
}

```

```

function filtrarProductos(
    lista,
    textoBusqueda,
    categoria,
    favoritosIds = []
) {

```

```

const texto = textoBusqueda
  .toLowerCase()
  .trim();

return lista.filter(producto => {

  const coincideTexto =
    producto.title
      .toLowerCase()
      .includes(texto);

  let coincideCategoria = true;

  if (categoria === "favoritos") {

    coincideCategoria =
      favoritosIds.includes(producto.id);

  } else if (categoria !== "todas") {

    coincideCategoria =
      producto.category === categoria;
  }

  return coincideTexto && coincideCategoria;
});
}

```

```

function calcularTotalCatalogo(lista) {

  return lista.reduce(
    (total, producto) => total + producto.price,
    0
  );
}

```

```

export {
  crearTarjetaProductoHTML,
  filtrarProductos,

```

```
    calcularTotalCatalogo  
};
```

3. ¿Qué hicimos exactamente?

Ahora viene la parte importante para que **entiendas el trabajo y no solamente copies el código**.

Tu profesor ya dejó indicadas las tecnologías y conceptos que quiere que practiquen: **ES Modules, DOM, eventos, fetch, async/await, Promise.all, localStorage, JSON, filter, map, reduce y Template Literals**. Eso se ve directamente en las consignas de los cuatro módulos.

A. app.js — El cerebro de la aplicación

app.js es el archivo que **coordina todo**.

Primero importamos las funciones de los otros módulos:

```
import { descargarProductosTech } from "./api.js";
```

Esto significa:

"Quiero utilizar la función descargarProductosTech que está dentro de api.js."

Después:

```
import {  
    obtenerFavoritos,  
    alternarFavorito  
} from "./storage.js";
```

Traemos las funciones encargadas de los favoritos.

Y finalmente:

```
import {  
    crearTarjetaProductoHTML,  
    filtrarProductos,
```

```
    calcularTotalCatalogo
} from "./ui.js";
```

Traemos las funciones relacionadas con la interfaz.

Esto corresponde directamente al primer TODO que dejó el profesor.

B. Seleccionar elementos HTML

Por ejemplo:

```
const btnTema = document.querySelector("#btn-tema");
```

Busca en el HTML:

```
<button id="btn-tema">
```

y guarda ese elemento en `btnTema`.

Entonces JavaScript puede hacer cosas como:

```
btnTema.addEventListener(...)
```

El profesor justamente dejó preparado el HTML con esos `id`, como `btn-tema`, `input-buscador`, `contenedor-catalogo`, etc.

C. Estado global

Tenemos:

```
let productosEnMemoria = [];  
let categoriaActual = "todas";
```

`productosEnMemoria` va a guardar todos los productos que descargamos.

Por ejemplo, imaginariamente:

```
[  
  producto1,  
  producto2,
```

```
    producto3,  
    producto4  
]
```

Mientras que:

```
categoriaActual = "todas";
```

indica qué filtro está seleccionado.

D. Favoritos

Esta función:

```
function actualizarBadgeFavoritos() {  
  
    const favoritos = obtenerFavoritos();  
  
    badgeFavoritos.textContent = favoritos.length;  
}
```

Primero obtiene los favoritos:

```
const favoritos = obtenerFavoritos();
```

Supongamos que devuelve:

```
[1, 5, 8]
```

Entonces:

```
favoritos.length
```

da:

```
3
```

Y finalmente:

```
badgeFavoritos.textContent = favoritos.length;
```

pone 3 en el contador del encabezado.

E. Filtros

Esta es una de las funciones más importantes:

```
function aplicarFiltros()
```

Primero obtenemos lo escrito en el buscador:

```
const texto = inputBuscador.value.trim();
```

Después obtenemos los favoritos:

```
const favoritosIds = obtenerFavoritos();
```

Y finalmente:

```
const productosFiltrados = filtrarProductos(  
    productosEnMemoria,  
    texto,  
    categoriaActual,  
    favoritosIds  
);
```

Le estamos diciendo a `ui.js`:

"Tomá todos mis productos y devolveme solamente los que coincidan con lo que estoy buscando."

F. `.filter()`

En `ui.js` usamos:

```
return lista.filter(producto => {
```

`filter()` sirve para **filtrar una lista**.

Por ejemplo:

```
const numeros = [1, 2, 3, 4, 5];
```

```
const resultado = numeros.filter(numero => numero > 3);
```

Resultado:

```
[4, 5]
```

En nuestro caso hacemos algo parecido, pero con productos.

G. Buscador

Tenemos:

```
const texto = textoBusqueda
    .toLowerCase()
    .trim();
```

Esto permite que:

iPhone

y:

IPHONE

sean tratados de la misma manera.

Después:

```
const coincideTexto =
    producto.title
        .toLowerCase()
        .includes(texto);
```

`includes()` comprueba si el nombre contiene el texto buscado.

Por ejemplo:

```
"iPhone 15".includes("iphone")
```

da:

true

H. Categorías

El HTML tiene:

```
<button data-categoria="smartphones">
```

En JavaScript podemos obtener ese valor mediante:

```
boton.dataset.categoria
```

Por eso hacemos:

```
const categoria = boton.dataset.categoria;
```

Si el botón es:

```
data-categoria="laptops"
```

entonces:

```
categoria
```

vale:

```
laptops
```

I. Favoritos dentro de filter()

Cuando seleccionamos:

Mis Favoritos

hacemos:

```
if (categoria === "favoritos") {  
  
    coincideCategoria =  
        favoritosIds.includes(producto.id);  
}
```

Por ejemplo, si tenemos:

```
favoritosIds = [2, 7, 10];
```

y el producto tiene:

```
producto.id = 7;
```

entonces:

```
favoritosIds.includes(7)
```

devuelve:

```
true
```

Por lo tanto aparece.

J. localStorage

El archivo `storage.js` se encarga de guardar los favoritos.

Usamos:

```
localStorage.setItem(  
    FAVORITOS_KEY,  
    JSON.stringify(favoritos)  
);
```

`localStorage` permite guardar información en el navegador.

Como `localStorage` guarda texto, convertimos el array:

```
[1, 4, 8]
```

en JSON mediante:

```
JSON.stringify()
```

Y cuando queremos recuperarlo:

```
JSON.parse()
```

Así los favoritos **no desaparecen simplemente por recargar la página**.

La consigna del profesor pide específicamente trabajar con LocalStorage & JSON.

K. API

En `api.js` tenemos tres URLs:

```
const urls = [  
  "https://dummyjson.com/products/category/smartphones",  
  "https://dummyjson.com/products/category/laptops",  
  "https://dummyjson.com/products/category/mobile-accessories"  
];
```

Son las tres categorías que pidió el profesor.

Después:

```
const respuestas = await Promise.all(  
  urls.map(url => fetch(url))  
);
```

Esto hace las tres solicitudes prácticamente en paralelo.

Es decir, en vez de:

Smartphones

↓

termina

↓

Laptops

↓

termina

↓

Accesorios

se solicitan juntas.

L. `async` / `await`

La función:

```
async function descargarProductosTech()
```

es asíncrona.

Por eso podemos utilizar:

```
await fetch(...)
```

`await` significa básicamente:

"Esperá a que termine esta operación antes de continuar."

Esto es necesario porque descargar información de Internet lleva tiempo.

M. `Promise.all()`

Tenemos:

```
const datos = await Promise.all(  
  respuestas.map(respuesta => respuesta.json())  
);
```

Esto espera a que todas las respuestas sean convertidas a JSON.

Después:

```
const productos = datos.flatMap(  
  datosCategoria => datosCategoria.products  
);
```

`flatMap()` nos permite juntar los productos de las tres categorías en **un único array**.

Por ejemplo:

```
[  
  {  
    products: [A, B, C]
```

```
    },  
    {  
      products: [D, E, F]  
    },  
    {  
      products: [G, H, I]  
    }  
  ]  
]
```

termina convertido en:

```
[A, B, C, D, E, F, G, H, I]
```

Eso coincide exactamente con lo que pide la consigna de `api.js`: descargar las tres categorías en paralelo y devolver un único array plano.

N. Crear las tarjetas

En `ui.js` utilizamos:

```
function crearTarjetaProductoHTML(producto, esFavorito)
```

Y extraemos los datos:

```
const {  
  id,  
  title,  
  price,  
  category,  
  thumbnail,  
  stock  
} = producto;
```

Esto se llama **destructuring**.

En vez de hacer:

```
producto.id  
producto.title  
producto.price  
producto.category
```

constantemente, obtenemos todas esas propiedades de una vez.

O. Template Literal

Después devolvemos HTML:

```
return `  
  <article class="tarjeta-producto">  
    ...  
  </article>  
`;  
`;
```

Las comillas invertidas:

```
`
```

permiten crear un **Template Literal**.

Y podemos insertar variables directamente:

```
${title}
```

Por ejemplo:

```
title = "iPhone 15"
```

produce:

```
<h3>iPhone 15</h3>
```

El profesor dejó indicada específicamente esta estructura para la tarjeta.

P. .map()

En app.js:

```
contenedorCatalogo.innerHTML = productosFiltrados  
  .map(producto => {  
  
    const esFavorito = favoritosIds.includes(producto.id);
```

```

        return crearTarjetaProductoHTML(
            producto,
            esFavorito
        );

    })
    .join("");

```

`map()` transforma cada producto en HTML.

Por ejemplo:

```

[
    producto1,
    producto2,
    producto3
]

```

se convierte en:

```

[
    "<article>...</article>",
    "<article>...</article>",
    "<article>...</article>"
]

```

Después usamos:

```

.join("")

```

para unir todo:

```

tarjeta1 + tarjeta2 + tarjeta3

```

Y finalmente lo colocamos en:

```

contenedorCatalogo.innerHTML

```

Q. `.reduce()`

Para calcular el valor total:

```
function calcularTotalCatalogo(lista) {  
  
    return lista.reduce(  
        (total, producto) => total + producto.price,  
        0  
    );  
}
```

Supongamos:

Producto A = \$100

Producto B = \$200

Producto C = \$50

reduce() hace:

$0 + 100 = 100$

$100 + 200 = 300$

$300 + 50 = 350$

Resultado:

350

Y lo mostramos:

```
totalPrecioSpan.textContent = `$$${totalPrecio.toFixed(2)}`;
```

quedando:

\$350.00

El uso de reduce() para sumar precios está pedido específicamente en la consigna.

R. Modo oscuro

El HTML ya tiene el botón:

```
<button id="btn-tema">
```

Y el CSS ya tiene preparado:

`body.dark-mode`

Entonces JavaScript solamente tiene que agregar o quitar esa clase:

```
document.body.classList.toggle("dark-mode");
```

Cuando la clase está:

```
<body class="dark-mode">
```

el CSS cambia los colores.

Cuando desaparece:

```
<body>
```

vuelve al modo claro.

El CSS del profesor ya tiene ambas paletas preparadas.

S. Delegación de eventos

Esta parte es bastante importante:

```
contenedorCatalogo.addEventListener("click", evento => {  
  
    const botonFavorito = evento.target.closest(".btn-fav-card");  
  
    if (!botonFavorito) {  
        return;  
    }  
  
    const id = Number(botonFavorito.dataset.id);  
  
    alternarFavorito(id);  
  
    actualizarBadgeFavoritos();  
  
    aplicarFiltros();  
});
```

¿Por qué no ponemos un `addEventListener` a cada estrella?

Porque las tarjetas son creadas **dinámicamente mediante JavaScript**.

En cambio, escuchamos los clicks desde:

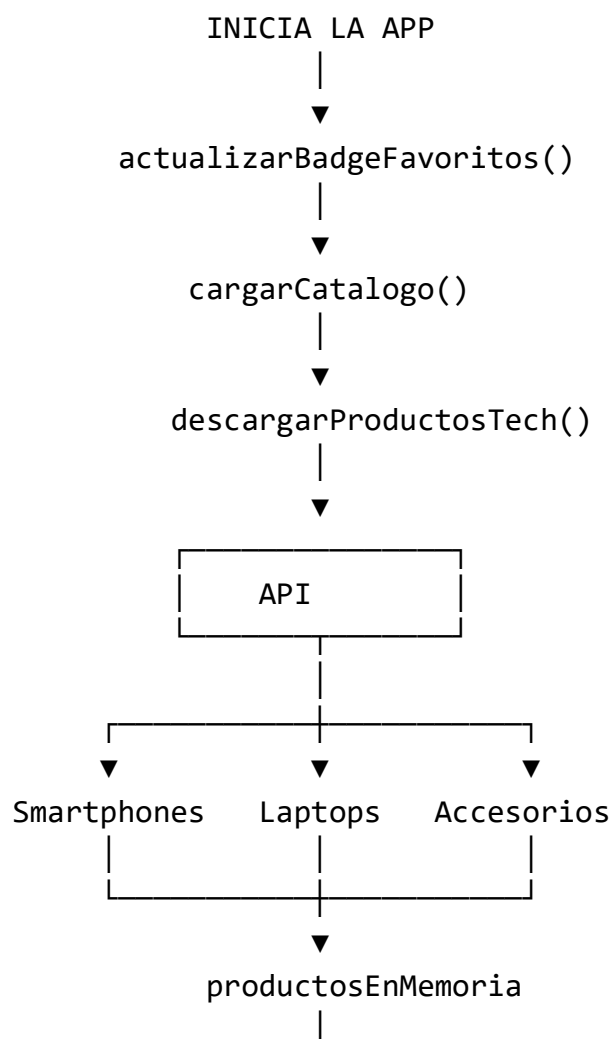
`contenedorCatalogo`

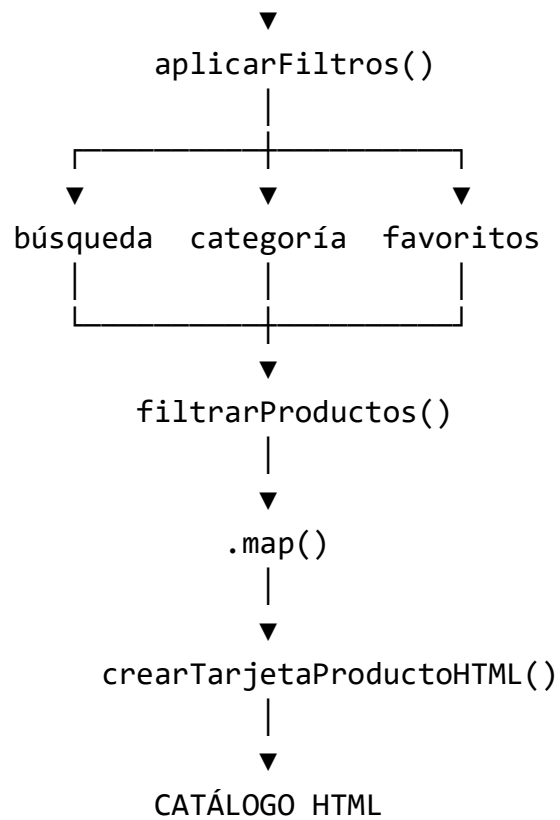
y averiguamos si el usuario realmente hizo click en un botón de favorito.

El profesor dejó expresamente esta tarea como **"Delegación de Favoritos en el Catálogo"**.

T. ¿Cómo funciona todo junto?

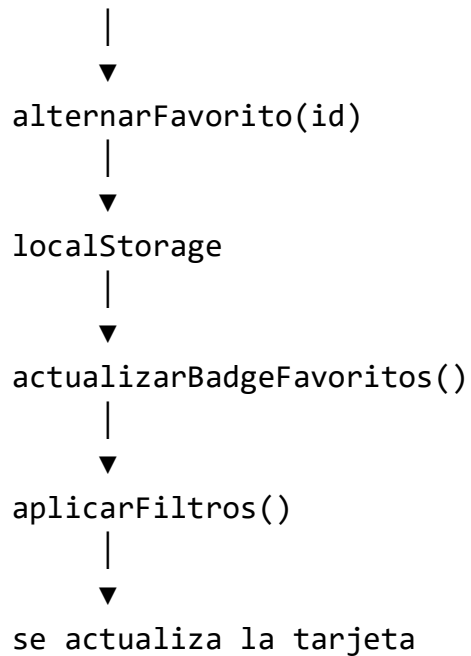
La aplicación termina funcionando aproximadamente así:





Y cuando el usuario toca una estrella:

Click en ★



Algo importante antes de probarlo

Tu `index.html` ya tiene:

```
<script type="module" src="./js/app.js"></script>
```

así que **no cambies eso**.

La estructura debería quedar así:

```
TU-PROYECTO/  
|  
├─ index.html  
├─ styles.css  
└─ js/  
    ├─ app.js  
    ├─ api.js  
    ├─ storage.js  
    └─ ui.js
```

Con esos cuatro archivos JS completados, el proyecto debería tener:

- carga de productos desde la API;
- smartphones, laptops y accesorios;
- buscador en tiempo real;
- filtros por categoría;
- favoritos;
- favoritos persistentes con `localStorage`;
- contador de favoritos;
- modo oscuro;
- cálculo de cantidad de productos;
- cálculo del valor total;
- spinner de carga;
- manejo de errores;
- botón de reintento;
- tarjetas generadas dinámicamente.

Y lo más importante para la entrega: **estás usando justamente los conceptos que el profesor dejó indicados en los TODO**, en lugar de hacer una solución completamente diferente.

Markdown.md pegado

Archivo

esta es la teoria de mi profe, quiero que te apoyes en eso y explica cada detalle del codigo,

Perfecto. Ahora sí tenemos las **dos piezas importantes**: el código del trabajo integrador y la teoría exacta de tu profesor. Voy a basarme principalmente en esa teoría, respetando sus nombres, conceptos y forma de explicarlos, en vez de explicarte JavaScript de una manera completamente distinta. La teoría está organizada en 5 bloques: DOM/eventos, HOFs y ES6+, asincronismo/Fetch, integración/persistencia/módulos y autoevaluación.

Además, hay algo importante: **el código que te pasé antes funciona como solución, pero ahora que veo la teoría, podemos explicarlo mucho mejor y detectar qué parte del trabajo corresponde a cada concepto que enseñó el profesor.**

Cómo vamos a estudiarlo

Te propongo hacerlo así:

Bloque 1 — DOM y eventos

- `document.querySelector()`
- `document.querySelectorAll()`
- `textContent`
- `innerHTML`
- `.value`
- `.dataset`
- `classList.add()`
- `classList.remove()`
- `classList.toggle()`
- `classList.contains()`
- `addEventListener()`
- `click`
- `input`
- `event.target`
- `closest()`
- delegación de eventos

Bloque 2 — Arrays, HOFs y ES6+

- `.forEach()`
- `.map()`
- `.filter()`
- `.includes()`
- `.reduce()`
- `.indexOf()`
- `.splice()`
- destructuring
- template literals
- arrow functions

Bloque 3 — API y asincronismo

- `fetch()`
- `async`
- `await`
- `Promise`
- `Promise.all()`
- `try...catch`
- `response.ok`
- `response.json()`
- `HTTP`

Bloque 4 — Integración

- `estados loading / éxito / error`
- `localStorage`
- `JSON.stringify()`
- `JSON.parse()`
- `import`
- `export`
- `arquitectura modular`

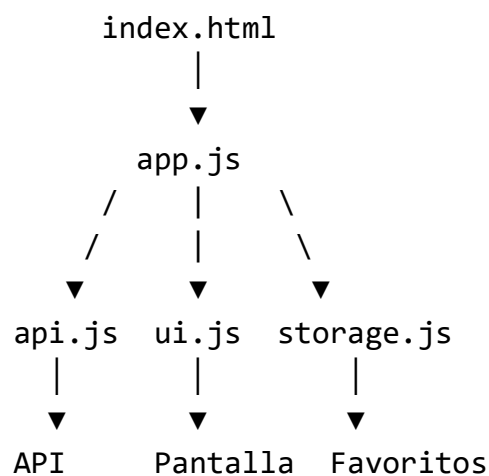
Esto coincide directamente con la estructura de la teoría.

1. Primero entendamos la arquitectura

Tu proyecto tiene aproximadamente esta estructura:

```
TechStore
├── index.html
├── styles.css
└── js/
    ├── app.js
    ├── api.js
    ├── storage.js
    └── ui.js
```

Y cada archivo tiene una responsabilidad diferente:



Esto es precisamente lo que tu profesor llama **modularización con ES Modules**: separar responsabilidades en distintos archivos y después conectarlos mediante `import` y `export`.

2. index.html: la estructura

Por ejemplo, tenemos:

```
<button id="btn-tema">
```

El HTML crea el elemento, pero **HTML por sí solo no hace que ese botón cambie el tema**.

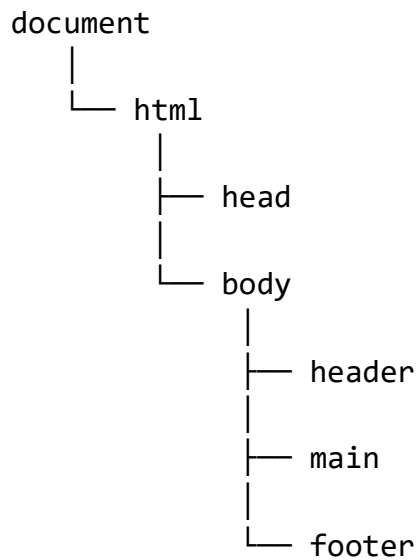
JavaScript posteriormente busca ese elemento:

```
const btnTema = document.querySelector("#btn-tema");
```

Acá aparece el primer concepto de la teoría: **DOM**.

Tu profesor explica que el navegador toma el HTML y construye en memoria un árbol de objetos. JavaScript trabaja sobre ese árbol, no modifica directamente el archivo `.html` que está guardado en el disco.

Podemos imaginarlo así:



3. `querySelector()`

Tenemos:

```
const btnTema = document.querySelector("#btn-tema");
```

Según la teoría:

`querySelector()` devuelve el primer elemento que coincida con el selector CSS.

El `#` significa que estamos buscando un `id`.

Por lo tanto:

```
document.querySelector("#btn-tema");
```

busca:

```
id="btn-tema"
```

y guarda ese botón en:

btnTema

Ahora podemos manipularlo.

4. querySelectorAll()

Tenemos:

```
const botonesFiltro =  
  document.querySelectorAll(".btn-filtro");
```

Acá hay una diferencia fundamental.

No estamos buscando **un botón**.

Estamos buscando **todos los botones** que tengan:

`.btn-filtro`

Por eso `querySelectorAll()` devuelve una **NodeList**, no un elemento individual.

Entonces hacemos:

```
botonesFiltro.forEach(boton => {  
  ...  
});
```

Esto también coincide exactamente con la advertencia que hizo tu profesor:

```
const botones =  
  document.querySelectorAll(".btn-comprar");  
  
botones.forEach(boton => {  
  boton.classList.add("activo");  
});
```

5. textContent

Tenemos:

```
badgeFavoritos.textContent = favoritos.length;
```

textContent sirve para colocar **texto plano** dentro de un elemento.

Por ejemplo, si:

```
favoritos.length
```

vale:

3

entonces el HTML pasa visualmente a mostrar:

★ 3

Es importante diferenciarlo de innerHTML.

6. innerHTML

En nuestro proyecto tenemos:

```
contenedorCatalogo.innerHTML = ...
```

Acá sí queremos insertar **HTML completo**, porque necesitamos generar las tarjetas.

Por ejemplo:

```
contenedorCatalogo.innerHTML = `  
  <article class="tarjeta-producto">  
    <h3>iPhone 15</h3>  
  </article>  
`;  
`;
```

innerHTML interpreta ese texto como HTML.

Tu profesor diferencia claramente:

textContent → texto plano

innerHTML → HTML estructurado

7. .value

Para el buscador:

```
const texto = inputBuscador.value.trim();
```

El HTML tiene:

```
<input
  type="text"
  id="input-buscador"
>
```

.value permite obtener lo que escribió el usuario.

Si escribe:

iphone

entonces:

```
inputBuscador.value
```

contiene:

```
"iphone"
```

Esto está relacionado con la teoría de lectura de atributos y valores del DOM.

8. data-* y .dataset

En las tarjetas tenemos:

```
<button
  class="btn-fav-card"
  data-id="15"
>
```

Ese:

```
data-id="15"
```

es un **atributo personalizado data-***.

JavaScript puede obtenerlo mediante:

```
botonFavorito.dataset.id
```

La teoría explica justamente que `data-*` sirve para asociar identificadores o metadatos a elementos y que se leen mediante `.dataset`.

Hay un detalle importante:

```
botonFavorito.dataset.id
```

devuelve texto:

```
"15"
```

Por eso hacemos:

```
const id = Number(botonFavorito.dataset.id);
```

y lo convertimos a número:

```
15
```

9. classList

Para el modo oscuro:

```
document.body.classList.toggle("dark-mode");
```

Esto está directamente relacionado con la teoría.

Tu profesor explica:

```
classList.add()  
classList.remove()  
classList.toggle()  
classList.contains()
```

En nuestro caso:

```
classList.toggle("dark-mode");
```

hace:

Si NO existe dark-mode



la agrega

Si YA existe dark-mode



la elimina

Por eso el CSS puede cambiar automáticamente.

10. Eventos

Tenemos:

```
btnTema.addEventListener("click", () => {
```

La teoría utiliza exactamente esta estructura:

```
elemento.addEventListener("evento", callback)
```

En nuestro caso:

elemento



btnTema

evento



click

callback



```
() => {
```

```
  ...
```

```
}
```

Es decir:

"Cuando el usuario toque el botón, ejecutá esta función."

11. Evento input

Nuestro buscador utiliza:

```
inputBuscador.addEventListener("input", () => {  
    aplicarFiltros();  
});
```

Esto significa que cada vez que el usuario escribe, se ejecuta `aplicarFiltros()`.

Por ejemplo:

Usuario escribe "i"



input



aplicarFiltros()

Usuario escribe "ip"



input



aplicarFiltros()

Usuario escribe "iph"



input



aplicarFiltros()

Es justamente el concepto de **búsqueda en tiempo real** que aparece en la teoría.

12. Delegación de eventos

Esta parte quiero que la entiendas especialmente bien porque probablemente el profesor la pregunte.

Tenemos muchas tarjetas:

Tarjeta 1 → ★
Tarjeta 2 → ★
Tarjeta 3 → ★
Tarjeta 4 → ★
Tarjeta 5 → ★
...

Podríamos poner un `addEventListener()` a cada estrella.

Pero la teoría enseña otra solución:

delegación de eventos.

Ponemos un único listener en:

```
contenedorCatalogo
contenedorCatalogo.addEventListener("click", evento => {
```

Cuando hacemos click en una estrella, el evento **burbujea** hacia los elementos padres.

Entonces usamos:

```
const botonFavorito =
  evento.target.closest(".btn-fav-card");
```

`event.target` representa el elemento donde comenzó el evento.

`closest()` busca hacia arriba hasta encontrar un elemento que coincida con:

`.btn-fav-card`

Si no encuentra ninguno:

```
if (!botonFavorito) {
  return;
}
```

Entonces no hacemos nada.

Esto coincide prácticamente línea por línea con el ejemplo de delegación de tu profesor.

13. .forEach()

En los botones:

```
botonesFiltro.forEach(boton => {
```

`forEach()` sirve para recorrer los elementos y realizar una acción.

La teoría lo define como:

recorrido para acciones, sin retorno.

Por eso:

```
botonesFiltro.forEach(boton => {  
    boton.addEventListener("click", ...);  
});
```

significa:

"Por cada botón de filtro, agregale un evento click."

14. .map()

Esta línea:

```
productosFiltrados.map(producto => {
```

es muy importante.

`map()` transforma cada elemento y devuelve un **nuevo array**.

En nuestro caso:

Producto 1

Producto 2

Producto 3

se transforma en:

HTML tarjeta 1
HTML tarjeta 2
HTML tarjeta 3

Después:

```
.join("")
```

une todas esas cadenas HTML.

Por eso tenemos:

```
productosFiltrados  
  .map(...)  
  .join("");
```

15. .filter()

En ui.js:

```
return lista.filter(producto => {
```

`filter()` devuelve solamente los elementos que cumplen una condición booleana.

Por ejemplo:

Todos:
iPhone
Galaxy
Laptop
Moto

Buscamos:

"iphone"

Entonces:

iPhone → true
Galaxy → false
Laptop → false
Moto → false

Resultado:

iPhone

16. .includes()

Tenemos:

```
favoritosIds.includes(producto.id)
```

La teoría explica que `includes()` permite verificar si un valor está dentro de un array de valores primitivos.

Por ejemplo:

```
const favoritos = [2, 5, 8];
```

```
favoritos.includes(5);
```

resultado:

true

Pero:

```
favoritos.includes(10);
```

resultado:

false

Por eso podemos determinar si una tarjeta es favorita.

17. .reduce()

Para el precio total:

```
return lista.reduce(  
  (total, producto) => total + producto.price,  
  0  
);
```

Tu profesor explica `reduce()` como:

acumulación en un único resultado.

Por ejemplo:

Producto 1 = 100

Producto 2 = 200

Producto 3 = 300

Tenemos:

$0 + 100 = 100$

$100 + 200 = 300$

$300 + 300 = 600$

Resultado:

600

18. Destructuring

En `ui.js` tenemos:

```
const {  
  id,  
  title,  
  price,  
  category,  
  thumbnail,  
  stock  
} = producto;
```

Esto se llama **destructuring**.

La teoría lo explica como una forma de extraer propiedades de un objeto en constantes individuales.

Si:

```
producto = {  
  id: 5,  
  title: "iPhone",
```

```
    price: 1000
  }
```

podemos hacer:

```
const { id, title, price } = producto;
```

Y obtenemos:

```
id = 5
title = "iPhone"
price = 1000
```

19. Template Literals

La tarjeta se construye con:

```
return `
  <article class="tarjeta-producto">
    ...
  </article>
`;
```

Las comillas invertidas permiten crear un **Template Literal**.

Dentro podemos insertar variables:

```
${title}
```

Por ejemplo:

```
const title = "iPhone 15";
```

entonces:

```
`${title}`
```

produce:

```
iPhone 15
```

Esto permite generar HTML dinámicamente.

20. API y fetch()

Ahora pasamos al tercer bloque.

Tenemos:

```
fetch(url)
```

Esto realiza una petición HTTP al servidor.

Tu teoría explica que una API REST trabaja mediante endpoints y que existe un ciclo:

```
Request
  ↓
Servidor
  ↓
Response
```

Nuestro programa solicita:

```
smartphones
laptops
mobile-accessories
```

a la API de DummyJSON.

21. ¿Por qué async?

Tenemos:

```
async function descargarProductosTech() {
```

La descarga de Internet no es instantánea.

JavaScript no debería congelar la interfaz mientras espera.

La teoría explica que las tareas lentas, como la red, se delegan a las Web APIs y que luego el Event Loop permite continuar el procesamiento cuando llegan los datos.

Por eso utilizamos asincronismo.

22. await

Tenemos:

```
const respuestas = await Promise.all(  
  urls.map(url => fetch(url))  
);
```

await significa:

"Esperá el resultado de esta operación asíncrona antes de continuar esta función."

Pero **no significa que congelemos toda la aplicación.**

Ese es justamente uno de los conceptos importantes del modelo de Event Loop que explica tu profesor.

23. Promise

Una petición como `fetch()` devuelve una Promise.

La teoría indica que una Promise puede estar:

pending
↓
fulfilled

o:

pending
↓
rejected

Es decir:

pending
= todavía esperando

fulfilled
= salió bien

rejected
= ocurrió un error

24. Promise.all()

Tenemos tres peticiones:

```
const urls = [  
  smartphones,  
  laptops,  
  accessories  
];
```

En lugar de hacer:

```
await fetch(smartphones);  
await fetch(laptops);  
await fetch(accessories);
```

una después de otra, utilizamos:

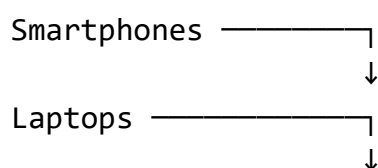
```
Promise.all(...)
```

para ejecutarlas en paralelo.

Esto es exactamente lo que recomienda la teoría para conexiones móviles con latencia.

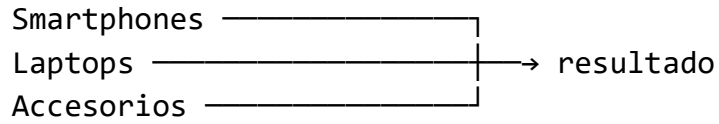
Visualmente:

SECUENCIAL



Accesorios —————┘

PARALELO



25. try...catch

En cargarCatalogo():

```
try {  
  
    productosEnMemoria =  
        await descargarProductosTech();  
  
} catch (error) {  
  
    console.error(  
        "Error al cargar el catálogo:",  
        error  
    );  
  
}
```

El try contiene el código que podría fallar.

Si ocurre un error, pasamos al:

catch

La teoría presenta precisamente `async / await` combinado con `try...catch` para manejar errores de operaciones asíncronas.

26. Algo MUY importante: `fetch()` y errores HTTP

En la teoría tu profesor aclara algo que probablemente sea una pregunta de examen:

`fetch()` no entra automáticamente en `catch` cuando recibe un 404 o 500.

Por eso en una versión completamente alineada con esa teoría conviene hacer:

```
if (!respuesta.ok) {  
  throw new Error(  
    `Error HTTP: ${respuesta.status}`  
  );  
}
```

Esto es justamente lo que agregamos en nuestro `api.js`.

27. `response.json()`

Después de recibir la respuesta:

```
const datos = await Promise.all(  
  respuestas.map(respuesta => respuesta.json())  
);
```

`response.json()` convierte el cuerpo de la respuesta en un objeto JavaScript.

Por ejemplo, el servidor podría devolver JSON:

```
{  
  "products": [  
    {  
      "id": 1,  
      "title": "iPhone"  
    }  
  ]  
}
```

y JavaScript puede trabajar con él como objeto.

La teoría muestra exactamente este paso mediante:

```
const data = await res.json();
```

28. localStorage

Los favoritos tienen que sobrevivir a una recarga.

Por eso tenemos:

```
localStorage.setItem(  
    FAVORITOS_KEY,  
    JSON.stringify(favoritos)  
);
```

La teoría explica:

```
JSON.stringify()  
    ↓  
objeto → texto
```

y:

```
JSON.parse()  
    ↓  
texto → objeto
```

Por ejemplo:

```
const favoritos = [1, 5, 8];
```

No guardamos directamente ese array como estructura JavaScript.

Lo convertimos:

```
JSON.stringify([1, 5, 8])
```

Resultado:

```
"[1,5,8]"
```

Eso se almacena en `localStorage`.

Después:

```
JSON.parse("[1,5,8]")
```

lo vuelve a convertir en:

```
[1, 5, 8]
```

29. import y export

En `api.js`:

```
export async function descargarProductosTech() {
```

Estamos diciendo:

"Esta función puede ser utilizada desde otro módulo."

Después en `app.js`:

```
import {  
  descargarProductosTech  
} from "../api.js";
```

Estamos diciendo:

"Quiero traer esa función desde `api.js`."

La teoría utiliza exactamente este concepto para explicar la modularización.

30. ¿Por qué el HTML tiene `type="module"`?

Tenemos:

```
<script  
  type="module"  
  src="../js/app.js">  
</script>
```

Esto le indica al navegador que `app.js` es un **módulo ES**.

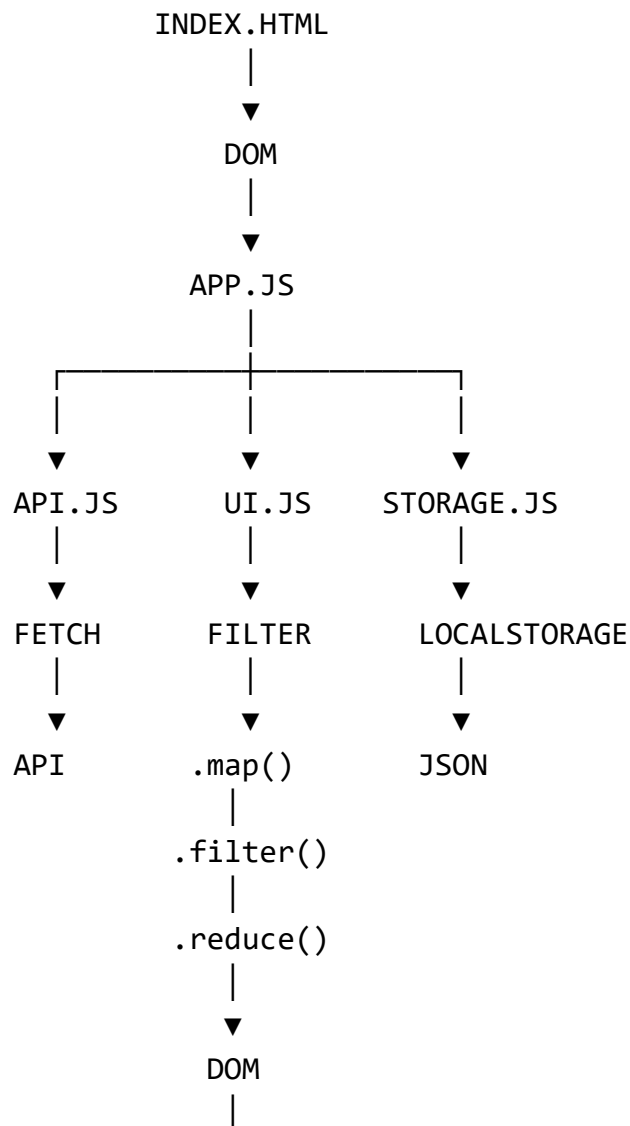
Y acá hay una advertencia importante de tu profesor:

No conviene abrir el HTML simplemente haciendo doble click.

Como estamos usando módulos, el proyecto debe servirse mediante un servidor local, por ejemplo **Live Server de VS Code**. La teoría explica que `type="module"` necesita que los archivos se sirvan mediante <http://> y no simplemente mediante <file:///>.

31. Cómo encaja TODO

Ahora podemos entender el proyecto entero:



▼
PANTALLA FINAL

Y cada bloque de la teoría tiene un lugar concreto:

Teoría del profesor	En nuestro proyecto
DOM	querySelector, querySelectorAll
textContent	contador y métricas
innerHTML	tarjetas
.value	buscador
dataset	ID del producto
classList	modo oscuro/filtros
addEventListener	botones y buscador
Event Bubbling	favoritos
Delegación	closest()
forEach	botones de categorías
map	tarjetas
filter	búsqueda/categorías
includes	favoritos
reduce	precio total
Destructuring	datos del producto
Template Literals	HTML de las tarjetas
fetch	API
async/await	descarga
Promise.all	3 APIs en paralelo
try/catch	errores
response.ok	errores HTTP
response.json()	datos de API
localStorage	favoritos
JSON.stringify	guardar favoritos
JSON.parse	recuperar favoritos
import/export	módulos

Así que **el trabajo integrador está diseñado para juntar prácticamente toda la teoría de la clase.**

Y algo que te recomiendo mucho: **no memorices el código entero.** Entendí qué problema resuelve cada parte. De hecho, las preguntas de autoevaluación que dejó tu profesor apuntan justamente a que puedas explicar por qué se usa cada técnica, especialmente querySelectorAll, mutación, HOFs, fetch/404 y Promise.all.

Si querés prepararte para defenderlo oralmente, el siguiente paso ideal es que hagamos **app.js línea por línea**, literalmente desde el `import` de la primera línea hasta `cargarCatalogo()`, explicando **qué hace, por qué se hace, qué teoría del profesor aplica y qué pasaría si sacamos esa línea**.